PROGRAM 10: RAG-Based Engineering Troubleshooting System

AIM

To implement a RAG-based engineering troubleshooting system that retrieves technical information and generates step-by-step recommendations.

ALGORITHM

1. Import the required libraries.
2. Load a pretrained embedding model.
3. Create a vector database.
4. Store technical troubleshooting documents.
5. Accept an engineering problem.
6. Retrieve relevant documents.
7. Generate troubleshooting recommendations.
8. Display the solution.

CODE

```
import chromadb

from sentence_transformers import SentenceTransformer

from transformers import pipeline


# Load models

embedding_model = SentenceTransformer(

    "all-MiniLM-L6-v2"

)


llm = pipeline(
```

```
    "text2text-generation",
    model="google/flan-t5-small"
)

# Create vector database
client = chromadb.Client()

collection = client.create_collection(
    name="troubleshooting_docs"
)

# Troubleshooting documents
documents = [
    "If a computer does not start, check the power cable and battery.",
    "If a computer is slow, close unnecessary programs and check RAM.",
    "If internet is not working, check Wi-Fi and restart the router.",
    "If a printer does not print, check paper and printer drivers.",
    "If a program crashes, check error messages and dependencies."
]

# Generate embeddings
embeddings = embedding_model.encode(
    documents
).tolist()
```

```
# Store documents
collection.add(
    documents=documents,
    embeddings=embeddings,
    ids=["1", "2", "3", "4", "5"]
)

# Get problem
problem = input("Describe your engineering problem: ")

# Retrieve relevant information
query_embedding = embedding_model.encode(
    [problem]
).tolist()

results = collection.query(
    query_embeddings=query_embedding,
    n_results=2
)

context = "\n".join(results["documents"][0])

# Generate recommendations
```

```
prompt = """
You are an engineering troubleshooting assistant.
Use the reference information to provide numbered steps.
Do not invent unsupported technical information.

Reference:
""" + context + """

Problem:
""" + problem

answer = llm(
    prompt,
    max_new_tokens=120
)

print("\nRetrieved Information:")
print(context)

print("\nTroubleshooting Recommendations:")
print(answer[0]["generated_text"])
```

INPUT

Describe your engineering problem:

My computer is connected to Wi-Fi but the internet is not working.

OUTPUT

Retrieved information:

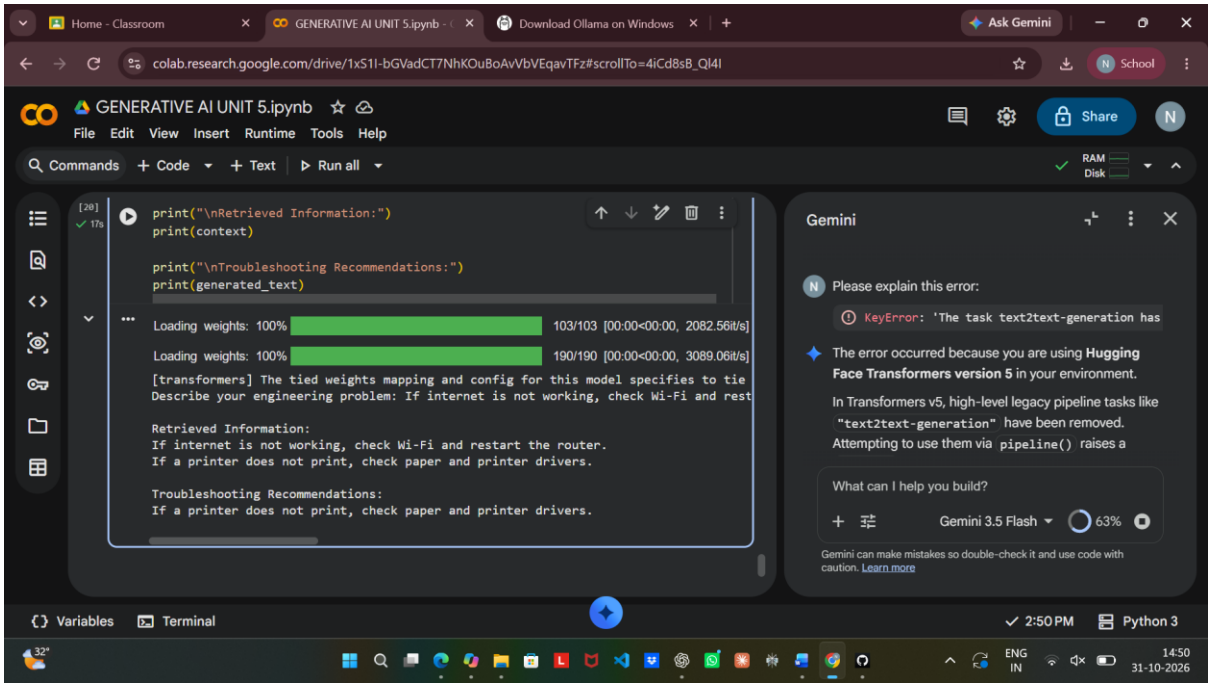
If internet is not working, check Wi-Fi and restart the router.

Troubleshooting recommendations:

1. Check the Wi-Fi connection.

2. Restart the router.

3. Check the internet connection again.



RESULT

Thus, a RAG-based engineering troubleshooting system was implemented using ChromaDB, Sentence Transformers and a pretrained language model.